RELATÓRIO TÉCNICO: Implementação do Módulo "Minhas Comissões" e Central de Relatórios

Projeto: Sistema de Gestão ERP e PDV (Projeto Integrador UNIVESP) **Arquitetura Base:** *Fat Models, Utility Services, Thin Views*

1. Resumo e Objetivo

Este relatório detalha o desenvolvimento e a integração do módulo "Minhas Comissões", projetado para automatizar o cálculo financeiro de remuneração variável da equipe de vendas. O objetivo do módulo é garantir precisão matemática no comissionamento sobre o valor líquido das vendas, prover transparência por meio de um extrato detalhado e aplicar Controle de Acesso Baseado em Funções (RBAC) para proteger dados sensíveis. Adicionalmente, foi implementada uma Central de Relatórios para otimizar a navegação (UX) do painel principal.

2. Regras de Negócio e Arquitetura (Como Funciona)

A arquitetura do sistema foi desenhada para blindar o cálculo financeiro contra flutuações futuras e erros operacionais:

- **Cálculo sobre o Valor Líquido:** A comissão incide estritamente sobre o valor final pago pelo cliente, após a dedução de descontos manuais e resgates do programa de pontos.
- **Imutabilidade Histórica:** O cálculo da comissão é realizado **no momento da venda**. O valor em reais é gravado permanentemente na nota fiscal interna (tabela `Vendas`), garantindo que se a taxa de comissão de um funcionário mudar no futuro, as vendas do passado não sejam recalculadas indevidamente.
- **Segurança (RBAC):** O sistema identifica automaticamente o usuário logado (via *Session*). Se o perfil for `Colaborador`, a rota de extrato exibe apenas as transações vinculadas ao seu próprio login, bloqueando tentativas de acesso via manipulação de URL. Se for `Administrador`, o acesso a todos os extratos é liberado.
- **Ciclo de Fechamento Mensal:** A view de processamento filtra nativamente as transações pelo mês e ano vigentes, "zerando" o painel automaticamente na virada do mês sem a necessidade de rotinas de limpeza de banco de dados (CRON jobs).

3. Metodologia: Passo a Passo da Implementação

Fase 1: Refatoração do Banco de Dados (Models) A tabela `Vendas` foi expandida com o campo `valor_comissao = models.DecimalField(...)`. Paralelamente, o método

`calcular_comissao` do modelo `Usuarios` (padrão *Fat Model*) foi reescrito para utilizar a biblioteca `Decimal` nativa do Python, garantindo precisão absoluta no tratamento de centavos e eliminando dízimas periódicas geradas por variáveis do tipo `float`.

Fase 2: Motor de Processamento (Services) O arquivo `pdv_services.py` foi atualizado. Dentro do bloco de transação atômica (`@transaction.atomic`), o sistema passou a capturar a taxa de comissão do vendedor logado, multiplicar pelo valor líquido do carrinho e gravar o resultado no novo campo da venda antes do *commit* no banco de dados. Em caso de cancelamento da venda, a comissão é estornada (zerada) na mesma trilha de auditoria.

Fase 3: Roteamento e Lógica de Apresentação (Views & URLs) Foi criada a nova rota `/gerencia/relatoriospainel/extrato/`. No arquivo `relatorios_views.py`, a função `tela_extrato_comissoes` foi implementada para receber parâmetros via `GET` (Mês, Ano e Vendedor). A função utiliza o ORM do Django para filtrar as vendas correspondentes, calcular as rubricas (Subtotal, Desconto Comum, Desconto por Pontos) e gerar um contexto iterável para o Front-end.

Fase 4: Interface de Usuário e Navegação (Front-end)

- **Central de Relatórios:** Criou-se o `relatorios_painel.html` atuando como um "hub" de navegação, ramificando o acesso entre "Desempenho da Loja" e "Minhas Comissões".
- **Tela de Extrato:** Desenvolveu-se o template `extrato_comissoes.html` com filtros interativos de data e uma tabela descritiva para exibir a matemática exata de cada cupom.
- **Botão Dinâmico:** Na tela de Gestão de Equipe, o saldo pendente foi transformado em um botão inteligente que embute o login do vendedor selecionado na URL de redirecionamento.

4. Facilidades Encontradas

- **Integração com o Motor do PDV Existente:** Como a arquitetura *Utility Services* já concentrava toda a lógica de salvamento no `pdv_services.py`, bastaram três linhas de código para injetar o cálculo da comissão no fluxo existente sem quebrar a criação de cupons ou baixas de estoque.
- **Agregações do Django ORM:** O uso de funções nativas do banco de dados, como `.aggregate(Sum('valor_comissao'))`, permitiu somar milhares de registros em milissegundos sem a necessidade de iterar sobre listas no Python (loops `for`), otimizando drasticamente o tempo de carregamento da tela de gestores.

5. Dificuldades Enfrentadas e Soluções

- **Desafio da Precisão Flutuante (Floating-Point Arithmetic):** Inicialmente, cálculos envolvendo centavos apresentavam discrepâncias devido ao comportamento nativo de pontos flutuantes na computação.

    - *Solução:* Conversão de todos os vetores financeiros para o tipo `Decimal` do pacote `decimal` do Python antes das operações algébricas.
- **Compatibilidade do Histórico de Dados (Legado):** Com a adição da coluna `valor_comissao`, as vendas realizadas antes da atualização ficaram com a comissão zerada por padrão no banco de dados (SQLite), causando inconsistência nos relatórios retroativos.
    - *Solução:* Desenvolvimento e execução de um *script* de migração direta no *shell* do Django (`python manage.py shell`), que iterou sobre todas as vendas com `status='VENDA'`, recalculou a comissão com base na taxa atual do vendedor e efetuou a atualização em massa (Data Backfill).

6. Conclusão

A implementação do módulo "Minhas Comissões" aliada à "Central de Relatórios" consolida a maturidade do sistema. A separação clara entre as regras de negócio no banco (imunidade a flutuações retroativas) e a exibição condicional nas views garante que o sistema atue como um ERP confiável e auditável, preparado para a escalabilidade de dados da JB Tintas.